Ders 7 Genişletilmiş Notu: Bağlı Listenin Değerlendirmesi, Yığın ve Aritmetik İfade Dönüşümleri

Ders 7 Genişletilmiş Notu: Bağlı Listenin Değerlendirmesi, Yığın ve Aritmetik İfade Dönüşümleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bağlı Listenin Değerlendirmesi
- 2. Yığın (Stack) Soyut Veri Tipi
- 3. Yığınla Parantez Dengesi Kontrolü
- 4. Diziyle Yığın Gerçekleştirme
- 5. Tek Yönlü Bağlı Listeyle Yığın Gerçekleştirme
- 6. Aritmetik İfadeler: Infix, Prefix ve Postfix
- 7. Infix'ten Postfix'e Dönüşüm
- 8. İki Yığınla İfade Değerlendirme
- 9. Yığındaki En Küçük Elemanı Takip Etme
- 10. Bir Diziyi İki Yığın Olarak Kullanma
- Örnek Sorular
- Dersin Toparlaması

1. Bağlı Listenin Değerlendirmesi

Hoca derse geçen haftanın kısa bir tekrarıyla başladı. Geçen hafta bağlı listenin (linked list) türleri konuşulmuştu. Tek yönlü bağlı listede (single linked list) ekleme (insertion), silme (deletion), arama ve güncelleme işlemlerinin nasıl yapıldığı, liste üzerinde nasıl gezinildiği, `head` ve `tail` kavramları görülmüştü. Tek yönde ilerlemek her zaman yetmediği için iki yönde ilerlemeyi sağlayan çift yönlü bağlı liste (double linked list) yapısı ele alınmıştı. Şarkılar ya da tarayıcıdaki sayfalar arasında ileri geri gitmek bunun tipik örneğidir. Listenin sonundan tekrar başına dönmek gerektiğinde dairesel bağlı liste (circular linked list) kullanılır. Düğümlerde hem önceki (`previous`) hem sonraki (`next`) adres varsa yapı dairesel çift yönlü bağlı listedir. Hoca, tek yönlü listedeki işlemleri diğer türler için de kendi denemelerimizle gerçeklememizi istemişti.

Bu derste bağlı liste konusu, avantajları, dezavantajları ve kullanım alanlarıyla bir bütün olarak değerlendirilip kapatıldı.

1.1 Avantajlar

- **Dinamik yapı:** Dizide olduğu gibi başta bir boyut vermek gerekmez. Eleman geldikçe bellek kullanılır, bu da belleğin boşa harcanmasını önler. Büyük verilerin söz konusu olduğu uygulamalarda, alan daha verimli kullanılsın diye bağlı liste tercih edilir.
- **Ekleme ve silmede kaydırma yok:** Bellekte elemanları sağa sola kaydırmak gerekmez. Yalnız bağlantı kırılıp yeni bir bağlantı kurulur. Hoca bunun en kötü durumda $O(1)$ kar-

maşıklık sağladığını, dizide ise ekleme ve silmenin $O(n)$ olduğunu söyledi. Bu yüzden ekleme ve silmenin çok olduğu durumlarda bağlı liste tercih edilir.
- **Esnek bellek kullanımı:** Bellekte arka arkaya bir milyon elemanlık yer yoksa ama farklı bölgelerde toplam bir milyon elemanlık yer varsa, bağlı liste bu dağınık yerleri kullanabilir.
- **Büyük veriye uygunluk:** Büyük veri büyük alan demektir. Arka arkaya bu kadar alan bulunamadığında küçük alanları birleştirerek iş çözülür.
- **Ölçeklenebilirlik:** Yapı istenildiği kadar genişletilebilir. Dizide ise yer bittiğinde genişletmek mümkün değildir.

  Ders sonrası not: Bağlı listede ekleme ve silmenin $O(1)$ olması, eklenecek ya da silinecek yerin adresi zaten biliniyorsa geçerlidir. Örneğin sıralı bir listede doğru yeri bulmak için önce liste üzerinde gezinmek gerekiyorsa işlemin toplam maliyeti $O(n)$ olur.

1.2 Dezavantajlar

- **Ek adres bilgisi:** Her eleman için bilginin yanında bir ya da iki adres saklanır. Bu, belleğin belli oranda fazla tüketilmesi demektir.
- **Doğrudan erişim yok:** Dizide bir elemana indisle doğrudan erişilir (direct/random access). Bağlı listede bir elemana, örneğin sondan bir önceki elemana ulaşmak için önceki bütün elemanların üzerinden tek tek geçmek gerekir.
- **Geriye gezinme zorluğu:** Dizide ileri geri hareket etmek kolaydır. Bağlı listede geriye doğru gezinmek ek adreslemeler gerektirir.
- **Arama verimsizliği:** Bir eleman aranırken düğümler tek tek atlanarak gezilir. Sıralı bir dizide ikili aramanın (binary search) avantajından yararlanılabilir, bağlı listede bu mümkün değildir.
- **Gerçekleştirme karmaşıklığı:** Dizi çok basit kullanılırken bağlı listede kendine referans veren düğümler oluşturmak, düğümü yaratmak, yer olup olmadığını kontrol etmek gibi farklı teknikler gerekir.
- **Veri paylaşımı zorluğu:** Dizide belli bir bölge kopyala-yapıştır mantığıyla blok halinde başka yere aktarılabilir. Bağlı listede elemanlar farklı yerlerde olduğu için her elemana tek tek ulaşıp değerini almak gerekir.
- **Küçük veri setlerinde avantaj yok:** Bu yüzden küçük verilerde genelde tercih edilmez.
- **Önbellek verimsizliği (cache inefficiency):** Hoca listenin sonunda duran bu maddenin belki en çok dikkat çekmesi gereken nokta olduğunu söyledi. Önbelleğe (cache) aktarım, belleğin belli bölümleri bir bütün halinde alınarak yapılır. Dizi bellekte bir arada durduğu için bir bölüm önbelleğe alındığında dizinin örneğin yarısı da alınmış olur ve o kısım üzerinde çok daha hızlı çalışılır. Bağlı listenin düğümleri belleğe dağıldığı için birbirini izleyen elemanlar bir bütün halinde önbelleğe aktarılamaz. Önbellek mimarisinin avantajı bağlı listede belli oranda kullanılamaz.

  Hocanın vurgusu: Önbellek verimsizliği, listenin en sonunda yer alsa da bağlı listenin en çok dikkat edilmesi gereken dezavantajlarından biridir.

  Ek bilgi (derste anlatılmadı): Önbelleğin bu davranışı “uzamsal yerellik” (spatial locality) ilkesine dayanır. Bir bellek adresine erişildiğinde, işlemci yalnız o baytı değil, onu içeren ve “önbellek satırı” (cache line) denen küçük bir bloğu getirir. Ardışık dizi elemanları aynı satırda bulunduğundan sonraki erişimler önbellekten karşılanır. Bağlı listede bir sonraki düğüm bellekte başka bir yerde olabildiği için her adımda yeni bir satır getirmek gerekebilir.

1.3 Kullanım Alanları

Hoca slaytta solda daha teorik kullanımları, sağda gerçek hayat uygulamalarını gösterdi.

Teorik kullanımlar:

- **Yığın ve kuyruk gerçekleştirme:** Yığın (stack) ve kuyruk (queue) hem diziyle hem bağlı listeyle gerçeklenebilir. İhtiyaca ve senaryoya göre ikisinden biri seçilir.
- **Ağaç ve graflar:** Düğümlerin birbirleriyle ilişkileri kurulurken ağırlıklı olarak bağlı liste yapısı kullanılacak.

- **Dinamik bellek ayırma (dynamic memory allocation).**
- **Seyrek matrislerin (sparse matrix) temsili:** Matrisin önemli bir bölümü boşsa ve yalnız belli gözlerde işe yarar eleman varsa buna seyrek matris denir. Örneğin 100×100'lük bir matriste yalnız 10 eleman varsa, bütün matrisi saklamak yerine yalnız bu elemanların satır, sütun ve değerleri saklanır. Bunun için bağlı listeden yararlanılabilir.
- **Büyük tam sayılarla aritmetik işlemler.**
- **Ağdaki eşler arasındaki ilişkiler**
- **İşletim sistemi:** Bellek yönetimi (memory management) ve süreç zamanlama (process scheduling). Hangi süreçten sonra hangisinin geleceği belirlenir, bütün süreçler bitince başa dönülür. Burada dairesel bağlı liste kullanılır.
- **Ekleme ve silmenin çok kullanıldığı bütün algoritmalar.**

Gerçek hayat uygulamaları:

- Müzik çalarda şarkılar, web tarayıcısında sayfalar, resim görüntüleyicide resimler arasında gezinme. İleri geri hareket edilen bütün senaryolarda, özellikle veri büyükse, bağlı liste kullanmak akıllıcadır.
- İşletim sisteminde en son kullanılan uygulamalar arasında geçiş (en son kullanılan, ondan öncekisi, ondan öncekisi).
- Cep telefonunda yeni bir kaydın rehbere eklenmesi.

Hoca, günün sonunda bilgisayar bilimlerinin uğraştığı birçok problemde dizi kadar bağlı listeyle de uğraşıldığını ve tasarımda bu yapının kullanılmasını önerdiğini söyledi.

1.4 Dairesel Bağlı Listenin Kullanım Alanları

Hoca konuyu kapatmadan önce dairesel bağlı liste örneklerini de ekledi:

- **Kuyruk:** Kuyruk gerçeklendiğinde her seferinde başa dönmek gerektiği için dairesel yapı kullanılır.
- **Medya oynatıcı:** Sondan baştaki videoya dönebilmek için.
- **Hash tablosu:** Tablonun sonuna gelince tekrar başa dönmek için. Bu, ilerleyen haftalarda görülecek.
- **Navigasyon sistemleri.**
- **Görev zamanlama (task scheduling):** İşletim sistemi örneğindeki gibi, süreçler arasında ilk sürece geri dönmek gerektiği için.

2. Yığın (Stack) Soyut Veri Tipi

2.1 LIFO Mantığı

Yığın bir soyut veri tipidir (abstract data type). Kuyrukta "ilk gelen ilk hizmeti alır" (first in first out, FIFO) kuralı geçerlidir. Bir bankaya gittiğinizde ilk gelen ilk hizmeti alır. Önceliklendirilmiş kuyruklar (priority queue) gibi özel türler de vardır. Yığın bunun tam tersidir: son giren ilk çıkar (last in first out, LIFO), başka bir deyişle ilk giren son çıkar (first in last out, FILO). Hoca, günlük hayatta kuyruğa daha çok rastladığımızı, ama yığının da önümüzdeki dönemde en az kuyruk kadar karşımıza çıkacağını söyledi.

2.2 Kullanım Alanları

- **Aritmetik ifadeler:** Kağıt üzerinde yazdığımız infix gösterimdeki ifadeyi (örneğin parantezli bir `A + B + C - ...` ifadesi) bilgisayar soldan sağa sırayla işlerse işlem önceliği doğru kurulmaz ve sonuç beklenenden farklı çıkar. Bu yüzden ifade postfix ya da prefix gösterime çevrilir. Bu dönüşümde yığın kullanılır.
- **Özyinelemeli çağrılar (recursive calls):** Bir fonksiyon başka bir fonksiyonu, o da başka bir fonksiyonu çağırıyorsa iç içe çağrılar oluşur. İlk çağrılan fonksiyon tabana yerleşir, sonraki onun üstüne, bir sonraki onun da üstüne. En son çağrılan fonksiyon ürettiği değeri bir öncekine aktarır, o da kendinden öncekine. Tabandaki ilk fonksiyon bütün sonuçları toplayıp nihai değeri döndürür. Yığın olmasaydı bunu kurgulamak kolay olmazdı.
- **Dizgi ters çevirme:** `ABCD` karakterleri sırayla yığına konup sırayla geri çekilince dizginin tersi (`DCBA`) elde edilir.

- **Geri izleme (backtracking):** Labirentte ilerlerken tıkanılan yollarda adım adım geri gidilir. Bu adım adım geri dönüş yığınla kolayca gerçeklenir.
- **Parantez kontrolü:** İfadelerde parantezler istemeden fazla ya da eksik yazılabilir. Bu bir sözdizimi (syntax) hatasıdır. Bunun otomatik kontrolünde yığın çok işe yarar.
- **Sayı tabanı dönüşümü:** Onluk sayıların başka sayı sistemlerine çevrilmesinde yığından yararlanılabilir.
- **Geri al (undo):** Hoca bunu ayrıca bir örnekle anlattı (Bölüm 2.4).

2.3 Yığın İşlemleri

Yığın iki yolla gerçeklenebilir: diziyle ya da bağlı listeyle. İkisi de mümkündür ve ikisi de kullanılacak.

Dizide ekleme, silme, güncelleme, arama ve doğrudan erişim vardır. Yığında ise temel işlemler şunlardır:

- `push`: yığına eleman koymak.
- `pop`: yığından eleman çıkarmak.
- `peek` ya da `top`: literatürde iki adla geçer. Yığın göstericisinin (stack pointer) o anda gösterdiği en üstteki değeri okur. Değer yalnız okunur, yığından çıkarılmaz.
- `isFull`: Yığının bir boyutu vardır. Büyüme şansı yoksa yığının dolup dolmadığı bununla kontrol edilir.
- `isEmpty`: O anda yığında eleman olup olmadığını kontrol eder.
- `size`: O anda yığındaki eleman sayısını döndürür.

Hocaya göre bunlardan en kritik olanları `push`, `pop`, `peek`, `isFull` ve `isEmpty`'dir.

2.4 Örnek: Metin Düzenleyicide Geri Al

Word ya da Notepad gibi bir editörde bir şeyler yazdığınızı, sonra yanlış yazdığınızı düşünüp bir kelimeyi sildiğinizi varsayın. Silinen kelime yığına atılır. Sonra ikinci bir kelime (slayttaki örnekte "very") silinir, o da yığına atılır. Geri al (undo) yapılınca otomatik olarak en son silinen kelime yerine gelir. Bir daha geri al yapılınca sıradaki kelime yerine yerleşir. Yığında 10 ya da 20 hamleyi tutacak kadar yer varsa bu hamleler arasında gezinilebilir. En son silinen ilk geri alınır, ilk silinen en son geri alınır. Buna yeniden yap (redo) da eklenebilir: geri alınan bir işlem tekrar yapılarak hamleler arasında gezinilir.

3. Yığınla Parantez Dengesi Kontrolü

Derleyicilerin (compiler) en sık yaptığı işlerden biri, verilen kodun sözdizimine uygun yazılıp yazılmadığını kontrol etmektir. Parantez kontrolü de en sık yapılan kontrollerden biridir. Açılan ve kapanan parantezlerin belli bir sayıda ve belli bir sırada olması gerekir.

Hoca bunun yığınla adım adım nasıl yapılacağını sordu. Bir öğrenci şu fikri verdi: her açılan parantez yığına itilir, karşılığı olan kapanan parantez geldiğinde bir tane çıkarılır, sonunda yığın boşsa sözdizimi hatası yoktur. Hoca bu fikri temel alıp beş örnek üzerinden kuralı tamamladı:

- `(` geldiğinde `push` yapılır.
- `)` geldiğinde `pop` yapılır. Yığın boşsa (`isEmpty`) çıkarılacak eleman yoktur, bu bir hatadır. Çünkü her kapanan parantezden önce bir açılan parantez gelmiş olmalıdır.
- İfade bittiğinde yığın boş değilse açılan-kapanan dengesi kurulmamıştır, bu da hatadır.

Örnek	İşlemler	Sonuç
`( ( (`	3 `push`, ifade biter	Yığın dolu kaldı, sözdizimi hatası
`)` ile başlıyor	Boş yığında `pop`	Hata
`( ( )`	2 `push`, 1 `pop`	Yığında 1 eleman kaldı, hata
`)` ile başlıyor	`isEmpty` doğru, `pop` yapılamaz	Hata
`( ( ) )`	2 `push`, 2 `pop`	Yığın boşaldı, hata yok

Hoca bunun çok basit bir örnek olduğunu, başka kombinasyonlarla da kurulabileceğini söyledi. Önemli olan açılan parantezleri `push` ile yığına koymak, kapanan parantez gelince `pop`

ile çıkarmak, `isEmpty` noktasında ya da işlem sonunda yığının boş olup olmadığına bakmaktır. Böylece başka bir veri yapısına gerek kalmadan sözdizimi kontrolü hızlıca yapılır.

Aşağıdaki program bu kuralı derste verilen örneklerle sınar (`gcc -Wall` ile derlenip çalıştırıldı; dördüncü örnek için `)(` kullanıldı):

```
#include <stdio.h>
#define MAX 100

struct stack { char item[MAX]; int top; };

int dengeli(const char *ifade)
{
    struct stack s;
    s.top = 0;
    for (; *ifade; ifade++) {
        if (*ifade == '(') {
            if (s.top == MAX)
                return 0;
            s.item[s.top++] = '(';      /* push */
        } else if (*ifade == ')') {
            if (s.top == 0)             /* boş yığında pop: hata */
                return 0;
            s.top--;                    /* pop */
        }
    }
    return s.top == 0;                  /* sonda yığın boş olmalı */
}

int main(void)
{
    const char *ornek[] = { "(((", ")", "(()", ")(", "(())" };
    int i;
    for (i = 0; i < 5; i++)
        printf("%-5s -> %s\n", ornek[i], dengeli(ornek[i]) ? "dogru" : "hata");
    return 0;
}
```

Çıktı: ilk dört örnek `hata`, `(())` ise `dogru`.

4. Diziyle Yığın Gerçekleştirme

4.1 Yapı ve İlklendirme

İlk adım bir yığın veri yapısı oluşturmaktır. `stack` adında bir yapı (structure) tanımlanır. İçinde bir dizi (`item`) ve bu dizinin, yani yığının ulaşabileceği en büyük boyut (`MAX`, örnekte 10) vardır. Ayrıca yığın göstericisinin nereyi gösterdiğini, kaçıncı elemanda olunduğunu tutan bir değişken gerekir: `top`. Bu iki elemanla bütün yığın yönetilir.

`initStack` fonksiyonuna yığının adresi gönderilir ve `top` 0 yapılır. Başlangıçta `top` 0. gözü gösterir ve içeride hiç eleman yoktur. Bundan sonra ya eleman gelecek ya da eleman çekilmek istenecek. Eleman geldiğinde yığının dolu olup olmadığı, eleman çekilmek istendiğinde boş olup olmadığı kontrol edilmelidir.

4.2 `isEmpty` ve `isFull`

`isEmpty`, `s->top`'un 0'a eşit olup olmadığına bakar. Eşitse yığın boştur ve 1 döner ("evet, boş"). 1'den 10'a kadar herhangi bir değerdeyse yığın boş değildir ve 0 döner.

`isFull`, `s->top`'un `MAX`'a eşit olup olmadığına bakar. Yığına yeterince eleman konduysa ve `top` artık 10'u gösteriyorsa 10 elemanlık yer dolmuştur, 1 döner. Değilse 0 döner.

> **Hocanın vurgusu:** Dolu bir yığına `push` yapılamaz. `push` sırasında `isFull`, `pop` sırasında `isEmpty` kontrolü mutlaka yapılmalıdır. Yoksa segmentation fault hatası oluşabilir.

Hoca, burada gösterilen yapının tek doğru yapı olmadığını, bu fonksiyonların farklı gerçekleştirmeleriyle karşılaşılabileceğini söyledi. Mantık aşağı yukarı aynıdır. Önemli olan, bir yığın kuruluyorsa bu fonksiyonların mutlaka kodlanmasıdır.

4.3 `push`

`push` iki parametre alır: yığının adresi ve eklenecek eleman `x`. Önce `isFull` çağrılır. 1 dönerse `if`'in doğru koluna girilir, `return 0` ile fonksiyon elemanı eklemeden biter, `push` başarısız olur. `isFull` 0 dönerse `else` koluna gidilir ve iki iş yapılır:

- `item` dizisinde `top`'un gösterdiği göze `x` yazılır.
- `top` bir artırılır ve bir sonraki gözü gösterir.

Bu iki satır `s->item[s->top++] = x;` biçiminde tek satırda da yazılabilir: önce `top`'un gösterdiği yere `x` yerleşir, sonra artırma işleciyle `top` bir artar. Arada hiçbir fark yoktur.

Fonksiyon sonunda `return 1` döner. Bu değer yığından alınan bir değer değildir, `push` işleminin başarılı olup olmadığını söyler: 0 başarısız, 1 başarılı demektir.

4.4 `pop`

`pop` da başarılı olup olmadığını döndürdüğü değerle bildirir. O halde çekilen elemanın değeri nerede saklanacak? Bunun için fonksiyona bir değişkenin adresi (`int *x`) gönderilir ve çekilen değer bu adrese yazılır. Böylece hem elemanın değeri alınır hem de işlemin başarısı kullanıcıya bildirilir.

İçeride eleman yoksa `pop` yapılamaz. Bu yüzden önce `isEmpty` kontrolü yapılır. Boşsa 0 döner, `pop` başarısız olur. Boş değilse `else` koluna gidilir. `push` elemanı koyduktan sonra `top`'u artırdığı için burada önce `top` azaltılır, sonra `item[top]`'taki değer `x`'in adresine yazılır ve 1 döner.

4.5 Adım Adım: `push 5`, `pop`, `push 7`

Bir öğrenci `top` kısmının tekrar anlatılmasını istedi. Hoca tahtada şu izi çizdi:

Adım	İşlem	`item[0]`	`top`	Açıklama
0	`initStack`		0	Yığın boş
1	`push 5`	5	1	`isFull` değil; `item[0] = 5`, sonra `top` 1 olur
2	`pop`	5	0	`isEmpty` değil; önce `top` 0 olur, sonra `item[0]` (5) `x`'e yazılır
3	`push 7`	7	1	5'in üzerine 7 yazılır, sonra `top` 1 olur

2. adımdan sonra 5 dizide fiziksel olarak durmaya devam eder. Hoca, orada bir şey yazıyor olmasının bir önemi olmadığını söyledi: `top` 0'ı gösterdiği için yığın boş kabul edilir. Sonraki `push`, aynı gözün üzerine yazar. Ana programda `x` yazdırılınca 5 görülür.

Bu gerçekleştirmede `top`'un değeri aynı zamanda yığındaki eleman sayısıdır (`size`). `top` 2 ise iki, 1 ise bir, 0 ise sıfır eleman vardır.

4.6 peek

`peek` ya da `top` (hoca daha çok `peek` adını kullandı), yığında örneğin 3, 5, 2 varsa en üstteki elemanın (2) değerini döndürür ama elemanı yığından çıkarmaz. `pop`'tan tek farkı budur.

Fonksiyona yine yığın ve değerin yazılacağı değişkenin adresi gönderilir. Bir değer okunacağı için `isEmpty` kontrolü yapılır, boşsa 0 döner. Değilse `pop`'taki gibi `top - 1` kullanılır, ama `top`'un kendisi değiştirilmez. `top - 1` ifadesi yalnız okunacak gözün indisini verir. O gözdeki eleman `x`'e yazılır ve 1 döner.

Temel fark şudur: `pop`'ta göstericinin değeri değiştirilir, `peek`'te göstericinin değeri yalnız kullanılır. Böylece en üstteki eleman yığında kalmaya devam eder.

4.7 Tam Kod

Hoca derste kodu slayt üzerinden anlattı. Aşağıdaki kod anlatımdan kurulmuştur ve `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırılmıştır.

```c
#include <stdio.h>
#define MAX 10

struct stack {
    int item[MAX];
    int top;
};

void initStack(struct stack *s)
{
    s->top = 0;
}

int isEmpty(struct stack *s)
{
    if (s->top == 0)
        return 1;
    return 0;
}

int isFull(struct stack *s)
{
    if (s->top == MAX)
        return 1;
    return 0;
}

int push(struct stack *s, int x)
{
    if (isFull(s))
        return 0;
    s->item[s->top] = x;   /* ya da tek satırda: s->item[s->top++] = x; */
    s->top++;
    return 1;
}

int pop(struct stack *s, int *x)
{
    if (isEmpty(s))
        return 0;
    s->top--;
    *x = s->item[s->top];
    return 1;
}
```

```c
int peek(struct stack *s, int *x)
{
    if (isEmpty(s))
        return 0;
    *x = s->item[s->top - 1];
    return 1;
}

int main(void)
{
    struct stack s;
    int x, i, n = 0;

    initStack(&s);
    push(&s, 5);
    printf("push 5 -> top = %d\n", s.top);
    pop(&s, &x);
    printf("pop -> x = %d, top = %d, item[0] = %d\n", x, s.top, s.item[0]);
    push(&s, 7);
    printf("push 7 -> top = %d, item[0] = %d\n", s.top, s.item[0]);
    pop(&s, &x);

    for (i = 1; i <= 11; i++)
        n += push(&s, i);
    printf("11 push denemesi, basarili: %d, top = %d, isFull = %d\n",
           n, s.top, isFull(&s));

    initStack(&s);
    push(&s, 3); push(&s, 5); push(&s, 2);
    peek(&s, &x);
    printf("peek -> %d, top = %d\n", x, s.top);
    initStack(&s);
    printf("bos yiginda pop: %d\n", pop(&s, &x));
    return 0;
}
```

Çıktı:

```
push 5 -> top = 1
pop -> x = 5, top = 0, item[0] = 5
push 7 -> top = 1, item[0] = 7
11 push denemesi, basarili: 10, top = 10, isFull = 1
peek -> 2, top = 3
bos yiginda pop: 0
```

Çıktı, hocanın izini doğrular: `pop` sonrasında 5 dizide durur ama `top` 0'dır. `push 7` aynı göze yazar. 10 elemandan sonra `push` başarısız olur ve `top` 10'da kalır. `peek` `top`'u değiştirmez.

Ders sonunda hoca bir yığının sahip olması gereken temel fonksiyonları tekrar sıraladı: `initStack`, `isEmpty`, `isFull`, `push`, `pop`, gerekirse `size` ve `peek`.

4.8 Sorular

Soru: `top` 0'dan başlıyorsa alabileceği en büyük değer 9 olamaz mı?

Cevap: Hoca bu soruyu her yıl aldığını ve haklı bir soru olduğunu söyledi. Kimi gerçekleştirmeler `top`'u 0'dan, kimi 1'den başlatır. Asıl soru, `top`'un bir eleman konduğunda mı yoksa hiç eleman yokken mi 0 olduğudur. Bu gerçekleştirmede hiç eleman yokken `top` 0'dır, bir eleman konunca 1 olur. Böylece 1, 2, …, 10'a kadar eleman konabilir. 10. eleman da konunca `top` 10 olur ve artık eleman eklenemez. Şekildeki ok 0. gözü gösteriyor gibi dursa da aslında "burada hiç eleman yok" anlamına gelir.

Soru: `top` sıfırken `item` dizisinin içi de sıfır mı olur?

Cevap: `top` sıfırken `item`'da (geçerli) hiçbir şey yoktur.

> **Hocanın vurgusu:** Bu gerçekleştirme, türlerden yalnız biridir. Sınırlar, neyin önce neyin sonra yapıldığına göre değişebilir. İnternette farklı kaynaklarda başka kabuller görebilirsiniz. Kendi içinde doğru çalıştığı sürece onlar da kullanılabilir.

Soru: Fonksiyon başarıya göre 0 ya da 1 döndürüyor, çekilen sayı için de bir işaretçi istiyoruz. Bunun yerine çekilen sayıyı doğrudan `return` ile döndürsek sorun olur mu?

Cevap: Yığında 0 saklama ihtimali varsa, `return 0` geldiğinde bunun hata mı yoksa sayı mı olduğu anlaşılamaz. Bunun dışında bir sakıncası yoktur. Sayılar arasında 0 olmayacağı biliniyorsa böyle de yapılabilir.

5. Tek Yönlü Bağlı Listeyle Yığın Gerçekleştirme

Bir öğrenci şunu sordu: yığını bağlı listeyle gerçekleştirseydik, `top`'u geri indirirken bir önceki elemanı bilmemiz gerekeceği için çift yönlü bağlı liste mi gerekir, yoksa tek yönlü liste yeterli mi?

Hoca tahtada bir tek yönlü liste çizerek problemi birlikte ele aldı. Başta `head` `NULL`'ı gösteriyorsa listede eleman yoktur; dizideki kontrollerin aynısı burada da geçerlidir. `push` ile 3 gelsin. Tutulması gereken şey 3'ün adresidir. Sonra 4 gelsin. Şimdi önemli olan 4'ün nereye ekleneceğidir. İki seçenek var:

- **Sona eklemek:** Her yeni elemanı o an tutulan elemanın arkasına eklemek.
- **Başa eklemek:** Her yeni elemanı listenin başına eklemek.

Hoca hangisinin daha uygun olduğunu sordu. Öğrenci aşağıdakinin, yani başa eklemenin daha uygun olduğunu, öbür türlü her seferinde listenin sonuna kadar gitmek gerektiğini söyledi. Hoca bunu onayladı: sıra gözetilmiyorsa yeni elemanı doğrudan başa eklemek akıllıcadır. Sona ekleme yapılırsa 5 geldiğinde de 6 geldiğinde de sona gidilir. Bunun tek çözümü `tail` bilgisini tutmaktır. `tail` saklanırsa sondaki eleman bilinir ve yeni eleman onun arkasına eklenir. Hoca bunun da kullanılabilecek bir yaklaşım olduğunu, ikisinin de olabileceğini söyledi.

Başa ekleme seçildiğinde: elde 3 vardı ve gösterici 3'ün adresini gösteriyordu. 4 gelince gösterici 4'ü gösterir. Sonra 2 gelince o da başa alınır, `head` 2'ye bağlanır. İlk eklenen eleman (3) doğal olarak `NULL`'ı gösterir. Liste şöyledir:

```
head -> 2 -> 4 -> 3 -> NULL
```

Bu haliyle yığın yapısı kurulmuştur ve geri gitmek için çift yönlü listeye gerek yoktur. `pop` yapılacağı zaman `head`'in gösterdiği düğüm çekilir. Hocanın örneğinde bu düğüm 5000 adresindeydi. Çekilince bu bağlantı kırılır ve `head`, çekilen düğümün gösterdiği adrese (4000, yani 4'ün düğümü) bağlanır. Bir `pop` daha 4'ü çıkarır, `head` 3000 adresindeki 3'ü gösterir. Son `pop` 3'ü çıkarır. 3'ün `next`'i `NULL` olduğu için `head` de `NULL` olur, liste ve yığın boşalır.

Aynı öğrenci, yığından en son elemanı çıkarmak için çift yönlü listenin daha kolay olup olmayacağını da sordu. Hoca şu an aranan sorunun bu olmadığını, ilk sorunun "yığın tek yönlü listeyle gerçeklenebilir mi" olduğunu belirtti. En alttaki elemana dizide olduğu gibi burada da elemanlar tek tek `pop` edilerek ulaşılır. Çift yönlü listenin daha avantajlı olup olmayacağı ayrıca incelenip konuşulmalıdır. Hoca bu sorunun güzel bir nokta olduğunu söyledi: ilk bakışta tek yönlü listeyle olmazmış gibi görünür, ama eleman ekleme biçimine göre hikaye değişir.

Aşağıdaki kod bu yaklaşımı gerçekler (anlatımdan kuruldu, `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırıldı):

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
```

```c
int push(struct node **head, int x)
{
    struct node *yeni = malloc(sizeof(struct node));
    if (yeni == NULL)
        return 0;
    yeni->data = x;
    yeni->next = *head;     /* yeni düğüm başa eklenir */
    *head = yeni;
    return 1;
}

int pop(struct node **head, int *x)
{
    struct node *eski;
    if (*head == NULL)      /* isEmpty */
        return 0;
    eski = *head;
    *x = eski->data;
    *head = eski->next;     /* head bir sonraki düğüme bağlanır */
    free(eski);
    return 1;
}

int main(void)
{
    struct node *head = NULL, *p;
    int x;
    push(&head, 3); push(&head, 4); push(&head, 2);
    printf("head");
    for (p = head; p != NULL; p = p->next)
        printf(" -> %d", p->data);
    printf(" -> NULL\n");
    while (pop(&head, &x))
        printf("pop: %d\n", x);
    printf("bos yiginda pop: %d\n", pop(&head, &x));
    return 0;
}
```

Çıktı:

```
head -> 2 -> 4 -> 3 -> NULL
pop: 2
pop: 4
pop: 3
bos yiginda pop: 0
```

> **Ek bilgi (derste anlatılmadı):** Başa ekleme ve baştan çıkarma yalnız `head` göstericisini değiştirdiği için, listenin uzunluğundan bağımsız olarak sabit sayıda adımda biter. Bu yüzden hem dizi tabanlı hem bu bağlı liste tabanlı gerçekleştirmede `push`, `pop` ve `peek` $O(1)$ zamanlıdır. Bağlı liste sürümünde ayrıca bir `isFull` sınırı yoktur; yer ancak `malloc` başarısız olunca biter.

6. Aritmetik İfadeler: Infix, Prefix ve Postfix

6.1 Neden Dönüşüm Gerekir?

Aritmetik ifade değerlendirme (arithmetic expression evaluation) önemli bir konudur, çünkü işlem öncelikleri vardır. Çarpma ve bölme, toplama ve çıkarmaya göre önceliklidir. Parantez kullanılırsa en yüksek öncelik parantezindir. Bugün yazarken kullandığımız gösterim infix gösterimdir (infix notation).

Hoca örnek olarak `3 + 5 * 2 / 3 + 7` ifadesini verdi. Bilgisayar bunu okuduğu sırayla işleseydi:

- $3 + 5 = 8$
- $8 \cdot 2 = 16$
- $16/3 \approx 5{,}33$
- $5{,}33 + 7 \approx 12{,}33$

Bu, yalnız verilen sıraya göre işlem yapmak, yani bildiğimiz kuralları çiğnemek demektir. İşlem önceliğine göre:

- Parantez yok, önce $5 \cdot 2 = 10$.
- $10/3 \approx 3{,}33$.
- $3 + 3{,}33 + 7 \approx 13{,}33$.

İki sonuç farklıdır (python ile doğrulandı: $12{,}333\ldots$ ve $13{,}333\ldots$). Bu yüzden iki ek gösterim kullanılır: prefix ve postfix.

6.2 Üç Gösterim

- **Infix:** Operandlar solda ve sağda, operatör ortada: `A + B`.
- **Prefix:** Önce operatör, sonra operandlar: `+ A B`.
- **Postfix:** Önce operandlar, sonra operatör: `A B +`.

İkinci satırda (`A + B * C`) önce `B * C` yapılır, sonra sonuç `A` ile toplanır. Postfix'te önce `B C *` yazılır, sonra `A` başa gelir ve toplama en sona: `A B C * +`. Yani önce operandlar sıralarıyla dizilir, sonra operatörler işlem sıralarıyla dizilir. Bir gösterimde öncelikle operatörler, öbüründe öncelikle operandlar gelir.

Üçüncü satırda (`A + B * C + D`) `B * C` çarpılır, sonuç `A` ile, o sonuç da `D` ile toplanır.

Dördüncü satır (`A + B * C - D / E * F`) parantezsizdir ve artı, çarpı, eksi, bölü, çarpı içerir. Öncelik sırası: önce `B * C`, sonra `A + B * C`, sonra `D / E`, sonra bunun `F` ile çarpımı, en son iki taraftan gelen sonuçların farkı.

Infix	Prefix	Postfix
`A + B`	`+ A B`	`A B +`
`A + B * C`	`+ A * B C`	`A B C * +`
`A + B * C + D`	`+ + A * B C D`	`A B C * + D +`
`A + B * C - D / E * F`	`- + A * B C * / D E F`	`A B C * + D E / F * -`

Tablodaki prefix ve postfix karşılıkları python ile ifade ağacı kurularak, postfix karşılıkları ayrıca aşağıdaki C dönüştürücüyle doğrulandı.

Öncelik sırası: parantez en üstte, sonra çarpma ve bölme, sonra toplama ve çıkarma. Kaynaklarda bunlara bazen üs alma işleci de eşlik eder; derste o ayrıntıya girilmedi, bilinen dört işlem üzerinden ilerlendi.

Soru: Dördüncü örnekte ilk işlem `B * C`, ikincisi `A` ile toplanması yazılmış. İkinci işlem `D / E` olabilir mi?

Cevap: Olabilir, yanlış değil. `B * C` ve `D / E`'nin çevresinde toplama ve çıkarma olduğu için bu iki işlem birbirinden bağımsız yapılır. Hangisinin önce yapıldığı sonucu etkilemez.

Slaytta dönüşüm sırasında yığına itilen ve çekilen operatörler de gösteriliyordu: + itilmiş, * itilmiş, sonra * ve + çekilmiş, sonra - gelmiş, üstüne / gelmiş, … Bu, operatörlerin yığına eklenme kurallarının belirlenmesi gerektiğini gösteriyordu. Hoca bilgisayarın bu dönüşümü nasıl yaptığına geçti.

7. Infix'ten Postfix'e Dönüşüm

Bilgisayarlar genelde prefix yerine postfix'e dönüştürmeyi tercih eder, ama prefix'e dönüştürebilmek de önemlidir. Dönüşümde bir operatör yığını, `push` ve `pop` işlemleri ve öncelik sırası kullanılır. Hocaya göre amaç önce işlemi tamamlamak değil, dönüşümü yapan adımları takip etmektir.

7.1 Parantezli Örnek: `3 * (5 + 4 / 1) - 2`

Hoca ifadeyi kenara temiz yazdı ve operand ya da operatör gördükçe adım adım ilerledi:

- 3 bir operanddır, doğrudan çıktıya yazılır.
- * gelir, yığın boştur, * yığına konur.
- (gelir. Açılan parantez hepsinin üzerinde önceliğe sahip olduğu için yığına yerleştirilir.
- 5 çıktıya yazılır.
- + gelir ve yığına konur. Hoca burada +'nın parantezden yüksek önceliğe sahip olmadığını söyledi, ama açılan parantezin içinde olunduğu için + yığına itilir.
- 4 çıktıya yazılır.
- / gelir. Bölme artıdan zaten yüksek önceliklidir; açılan parantezin içinde olunduğu için yığına konur. Kapanan parantez görülene kadar gelen operatörler bu şekilde eklenmeye devam eder.
- 1 çıktıya yazılır.
-) gelir. Kural: açılan parantez görülene kadar `pop` yapmaya devam et. `push` yapılırken yığına, `pop` yapılırken postfix ifadesinin oluşturulduğu diziye yazılır. Önce / çekilip çıktıya yazılır. Parantez hâlâ görülmedi, bir `pop` daha yapılır, + çıktıya yazılır. Sonra (görülür, o da çekilir ama çıktıya yazılmaz. Yığında yalnız * kalır.
- - gelir. -, yığının tepesindeki *'dan daha öncelikli değildir. Başka bir deyişle * daha öncelikli olduğu için çekilip çıktıya yazılır. Yığın boşalınca - yığına konur.
- 2 çıktıya yazılır.
- İşlenecek eleman kalmadığı için yığın boşalana kadar `pop` yapılır, - çıktıya yazılır.

Okunan	İşlem	Yığın (alt → üst)	Çıktı
3	Çıktıya yaz		3
*	push	*	3
(	push	* (	3
5	Çıktıya yaz	* (	3 5
+	push (tepede ()	* (+	3 5
4	Çıktıya yaz	* (+	3 5 4
/	push	* (+ /	3 5 4
1	Çıktıya yaz	* (+ /	3 5 4 1
)	(görülene kadar pop	*	3 5 4 1 / +
-	* çekilir, - itilir	-	3 5 4 1 / + *
2	Çıktıya yaz	-	3 5 4 1 / + * 2
Son	Yığını boşalt		3 5 4 1 / + * 2 -

Sonuç postfix ifade `3 5 4 1 / + * 2 -` olur. Hoca bunun nasıl hesaplanacağını da kısaca söyledi: ilk adım $4/1$, sonuç 5 ile toplanır, o sonuç 3 ile çarpılır, en son 2 çıkarılır. Sayısal olarak: $4/1 = 4$, $5 + 4 = 9$, $3 \cdot 9 = 27$, $27 - 2 = 25$ (ara adım eklendi; C ve python ile doğrulandı).

> **Ders sonrası not:** Hoca hesabın son adımı için "çıkan sonucu 2'den çıkartacaktım" dedi. Doğrusu, sonuçtan 2'nin çıkarılmasıdır: $27 - 2 = 25$. Postfix'te `a b -`, $a - b$ demektir.

Hoca temel kuralları şöyle özetledi:

- Gelen operatör yığının tepesindekinden daha yüksek önceliğe sahipse doğrudan `push` edilir.
- Yığının tepesindeki operatör gelen operatörden daha yüksek önceliğe sahipse o `pop` edilir.
- Parantez açılınca, açılan parantezin içindeki işlemler kapanan parantez görülene kadar sürdürülür. Kapanan parantez görülünce açılan parantez görülene kadar `pop` yapılır.

7.2 Parantezsiz Örnek: `3 * 5 + 4 / 1 - 2`

Aradan sonra hoca aynı ifadeyi parantezsiz yazdı: `3 * 5 + 4 / 1 - 2`. Normalde $3 \cdot 5 = 15$, $4/1 = 4$, $15+4-2 = 17$ beklenir. Hiç işlem önceliği gözetilmeden sırayla yapılsaydı $3 \cdot 5 = 15$, $15 + 4 = 19$, $19/1 = 19$, $19 - 2 = 17$ olurdu. Bu örnekte iki yol tesadüfen aynı sonucu verir.

Hocanın tahtadaki adımları:

- 3 çıktıya yazılır, * yığına konur, 5 çıktıya yazılır.
- + gelir. Hoca ne olacağını sordu. Tepede * var ve * öncelikli olduğu için çekilip çıktıya yazılır. Ardından + itilir.
- 4 çıktıya yazılır.
- / gelir. Tepede + var. / daha öncelikli olduğu için +'nın üzerine itilir.
- 1 çıktıya yazılır.
- - gelir. Tepede / var. Bir öğrencinin cevabıyla / öncelikli olduğu için çekilip çıktıya yazılır. Sonra -, +'nın üzerine konur.
- 2 çıktıya yazılır. İfade bitince önce -, sonra + çekilir.

Hocanın çıktısı: `3 5 * 4 1 / 2 - +`. Hoca bu noktada slayttaki başlığı "prefix" yazdığını fark edip doğrusunun "postfix" olduğunu söyledi.

Ardından hoca, 3 ile 5'in çarpılacağını, 4 ile 1'in bölüneceğini, ortaya çıkan iki değerle hangi işlemin yapılacağını sordu. Bir öğrenci işleme devam edileceğini söyledi: 15 ve 4 elde edilir, 4'ten 2 çıkarılır, sonra en son toplama yapılır. Hoca bunu onayladı ve parantezin olup olmamasının önemli bir fark yarattığını söyledi. Açılan parantezden kapanan paranteze kadar olan bölüm özel değerlendirilir, çünkü birinci önceliğe sahiptir. Parantez yoksa tamamen işlem önceliklerine göre gidilir: elinizdeki işlem yığının tepesindekinden daha az öncelikliyse tepedeki çekilip sıraya konur; elinizdeki daha öncelikliyse doğrudan itilir ve yığının üstünde durur, daha öncelikli olduğu için de ilk o işlem görür.

> **Ders sonrası not:** Hoca - geldiğinde yalnız /'yi çekip -'yi +'nın üzerine itti ve kuralı "tepedeki operatör daha yüksek öncelikliyse çek" biçiminde (kesin büyüklükle) anlattı. Doğru kural, tepedeki operatörün önceliği gelen operatörün önceliğine **büyük ya da eşit** olduğu sürece çekmeye devam etmektir. + ve - eşit önceliklidir ve soldan sağa işlenir, bu yüzden + da çekilmelidir. Doğru postfix `3 5 * 4 1 / + 2 -`'dir. Hocanın çıktısı `3 5 * 4 1 / 2 - +`, yani $15 + (4 - 2)$ ifadesidir. Bu örnekte ikisi de 17 verir (C ile doğrulandı), ama genel olarak aynı değildir: `A - B + C` ifadesine hocanın kuralı uygulanırsa `A B C + -`, yani $A - (B + C)$ çıkar; doğru postfix `A B - C +`'dır. Derste öğrencinin söylediği "önce 4'ten 2 çıkar, sonra topla" hesabı da hocanın çıktısını izler; sonucun doğru çıkması bu örneğe özgüdür.
>
> **Ek bilgi (derste anlatılmadı):** Eşit öncelikli işleçlerin hangi sırayla bağlandığına birleşme yönü (associativity) denir. Dört işlemin hepsi soldan birleşimlidir: `A - B + C`, $(A - B) + C$ demektir. Üs alma gibi sağdan birleşimli işleçlerde ise eşit öncelikte çekme yapılmaz, yalnız tepedeki kesin daha öncelikliyse çekilir. Derste üs alma kullanılmadığı için "büyük ya da eşitse çek" kuralı yeterlidir.

Dönüşüm kuralının tamamı (ders sonrası düzeltmeyle) şöyle yazılabilir:

- Operand doğrudan çıktıya yazılır.
- `(` her zaman yığına itilir.
- `)` gelince `(` görülene kadar `pop` yapılıp çıktıya yazılır; `(` atılır.
- Operatör gelince, yığının tepesinde `(` olmayan ve önceliği gelen operatörünkinden büyük ya da eşit bir operatör olduğu sürece `pop` yapılıp çıktıya yazılır, sonra gelen operatör itilir.
- İfade bitince yığın boşalana kadar `pop` yapılıp çıktıya yazılır.

8. İki Yığınla İfade Değerlendirme

Hoca, derleyicilerin ve bilgisayarların bu işi gerçek hayatta nasıl yaptığına geçti. Dönüşümü ve sonucu hesaplamayı aynı anda yaparlar. Bunun için iki yığın kullanılır: biri operand yığını, diğeri operatör yığını. Bir önceki yöntemde çıktı ayrı bir dizide saklanıyordu. Burada iki yığın vardır ve derleyici ifadenin sonucunu bu iki yığın yardımıyla elde eder.

İfade: `5 * (4 / 2 + 3 * 6) - 7`.

Kurallar: operand görülünce operand yığınına itilir. Bir operatör çekildiğinde operand yığınından iki sayı çekilir. İlk çekilen sayı işlemin ikinci (sağ) kısmına, ikinci çekilen sayı ilk (sol) kısmına konur. Yani operand yığınından iki `pop`, operatör yığınından bir `pop` yapılır. Sonuç tekrar operand yığınına itilir.

Adımlar:

- 5 operand yığınına; * ve (operatör yığınına; 4 operand yığınına; / operatör yığınına; 2 operand yığınına itilir.
- + gelir. Hoca pop mu push mu olacağını sordu. Bir öğrenci pop olacağını, ama sayıların da çıkarılması gerektiğini söyledi. pop olmasının sebebi, elimizdeki +'nın tepedeki /'den daha az öncelikli olmasıdır: önce bölme yapılmalıdır. / çekilir. Operand yığınından önce 2, sonra 4 çekilir. $4/2 = 2$ sonucu operand yığınına itilir. Operand yığınında 5 ve 2 vardır. Sonra + operatör yığınına itilir.
- 3 operand yığınına, * operatör yığınına, 6 operand yığınına itilir. Şu an operand yığınında dört (5, 2, 3, 6), operatör yığınında dört (*, (, +, *) eleman vardır.
-) gelir. (görülene kadar işlemler yapılır. Önce en üstteki * çekilir. İlk çekilen 6 sağa, ikinci çekilen 3 sola yazılır: $3 \cdot 6 = 18$. 18, 3 ve 6'nın boşalttığı yere itilir. Operand yığını 5, 2, 18 olur.
- Sırada + vardır. Çekilir, iki operand gerekir: önce 18, sonra 2 çekilir. $2 + 18 = 20$ itilir. Operand yığınında 5 ve 20 kalır. Parantezle de iş bitmiştir, (atılır. Operatör yığınında * kalır.
- - gelir. * daha öncelikli olduğu için çekilir. İlk çekilen 20, ikinci çekilen 5: $5 \cdot 20 = 100$, 100 itilir. Sonra - operatör yığınına, 7 operand yığınına itilir.
- İfade biter. Kalan işlemler yapılır: - çekilir, ilk değer 7, ikinci değer 100: $100 - 7 = 93$. 93 itilir.
- Operatör yığını boştur ve işlenecek eleman kalmamıştır. Operand yığınındaki değer çekilip ekrana yazdırılır: 93.

Okunan	Operand yığını	Operatör yığını	Yapılan işlem
5	5		
*	5	*	
(	5	* (	
4	5 4	* (	
/	5 4	* (/	
2	5 4 2	* (/	
+	5 2	* (+	$4/2 = 2$
3	5 2 3	* (+	
*	5 2 3	* (+ *	
6	5 2 3 6	* (+ *	
)	5 20	*	$3 \cdot 6 = 18, 2 + 18 = 20$
-	100	-	$5 \cdot 20 = 100$
7	100 7	-	
Son	93		$100 - 7 = 93$

Aşağıdaki kod bu yöntemi tek basamaklı sayılar için gerçekler (anlatımdan kuruldu, `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırıldı). İşlem sırası hocanın tahtadaki sırasıyla aynı çıkar:

```c
#include <stdio.h>
#include <ctype.h>
#define MAX 100

int opnd[MAX], ntop = 0;      /* operand yığını */
char oper[MAX]; int otop = 0;  /* operatör yığını */

int oncelik(char op)
{
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

void islemYap(void)
{
    char op = oper[--otop];
    int b = opnd[--ntop];      /* ilk çekilen: ikinci (sağ) operand */
```

```
    int a = opnd[--ntop];      /* ikinci çekilen: birinci (sol) operand */
    int r = 0;
    switch (op) {
    case '+': r = a + b; break;
    case '-': r = a - b; break;
    case '*': r = a * b; break;
    case '/': r = a / b; break;
    }
    printf("  %d %c %d = %d\n", a, op, b, r);
    opnd[ntop++] = r;
}

int hesapla(const char *s)
{
    for (; *s; s++) {
        char c = *s;
        if (isdigit((unsigned char)c)) {
            opnd[ntop++] = c - '0';
        } else if (c == '(') {
            oper[otop++] = c;
        } else if (c == ')') {
            while (oper[otop - 1] != '(')
                islemYap();
            otop--;                          /* '(' atılır */
        } else {
            while (otop > 0 && oncelik(oper[otop - 1]) >= oncelik(c))
                islemYap();
            oper[otop++] = c;
        }
    }
    while (otop > 0)
        islemYap();
    return opnd[--ntop];
}

int main(void)
{
    printf("5*(4/2+3*6)-7\n");
    printf("sonuc = %d\n", hesapla("5*(4/2+3*6)-7"));
    return 0;
}
```

Çıktı:

```
5*(4/2+3*6)-7
  4 / 2 = 2
  3 * 6 = 18
  2 + 18 = 20
  5 * 20 = 100
  100 - 7 = 93
sonuc = 93
```

> **Hocanın vurgusu:** İlk çekilen operand işlemin ikinci kısmına, ikinci çekilen operand birinci kısmına konur. Çıkarma ve bölmede bu sıra sonucu değiştirir.

8.1 Sorular

Soru: Önceki slaytta postfix gösterimin yanında “15 + 4 = 19, 19 / 1” diye bir hesap var. 4 / 1 olması gerekmiyor mu, bunun postfix ile ilgisi var mı, yoksa bir hata mı?

Cevap: O hesap, bilgisayar öncelik sırası gözetmeden infix ifadeyi adım adım işleseydi ne olacağını göstermek içindi: $3 \cdot 5 = 15$, $15 + 4 = 19$, … Bu örnekte sonuçlar tesadüfen aynı

çıktı. Hoca farkı görmek için ifadeyi değiştirdi: `2 * 4 - 8 / 2 + 3`. İşlem önceliğine göre $8 - 4 + 3 = 7$ olur. Sırayla yapılsaydı $2 \cdot 4 = 8$, $8 - 8 = 0$, $0/2 = 0$, $0 + 3 = 3$ olurdu. Doğru sonuç 7 iken öncelik gözetilmeseydi 3 çıkacak, yani yanlış olacaktı (python ile doğrulandı).

Soru: Bu işlemin akış diyagramını ya da kodunu gösterecek misiniz?

Cevap: Hayır. Pekiştirme ve alıştırma için bunu öğrencilerin kendisi yapmalıdır. `pop` ve `push` fonksiyonlarını gerçeklemek ayrı bir iştir; bir akış diyagramında bazı adımlar doğrudan `pop` ve `push` olarak ifade edilebilir, bunda sakınca yoktur. Hoca bunun gerçekten güzel bir problem olduğunu, haftaya kadar uğraşıp takılan olursa birlikte bakılabileceğini söyledi. (Bu notta bir çözüm, Örnek Sorular bölümünde verilmiştir.)

9. Yığındaki En Küçük Elemanı Takip Etme

Hoca yeni bir problem verdi. Bir yığına sırayla 3, 5, 1, 2, 4 gibi elemanlar itiliyor; zaman zaman `pop`, zaman zaman `push` yapılıyor. Herhangi bir T anında yığındaki elemanlar arasındaki en küçük değeri bulmamız istenseydi ne yapardık? İpucu: ek bir yardımcı yığın (auxiliary stack) kullanma hakkımız var. Başta yığın boştur, dolu düşünülmemeli; elemanlar `push` ile sırayla gelir, arada `pop` ile çıkabilir.

Öğrencilerle birlikte şu çözüm kuruldu:

- 3 gelir. Yardımcı yığın boştur ve 3 o anki en küçük değerdir, yardımcı yığına da itilir.
- 5 gelir. 3 ile karşılaştırılır, büyük olduğu için yardımcı yığına itilmez.
- 1 gelir. 3'ten küçük olduğu için yardımcı yığına itilir.
- 2 gelir, küçük değildir, itilmez. 4 de itilmez.
- -5 gelir, küçük olduğu için yardımcı yığına itilir.

Yardımcı yığının tepesindeki eleman her zaman o anki en küçük elemandır: ana yığın 3, 5, 1, 2, 4, -5; yardımcı yığın 3, 1, -5; en küçük -5.

Sonra hoca, ana yığından `pop` yapılırsa ne olacağını sordu. Tepedeki -5 çıkınca en küçük 1 olmalıdır, ama bunu nasıl bileceğiz? Tahtada gördüğümüz için biliyoruz, ama program yardımcı yığının durumunu "görmez". Bir öğrenci karşılaştırınca yer değiştirmeyi önerdi; hoca yığında yalnız `pop` ve `push` yapılabileceğini hatırlattı. Başka bir öğrenci doğru fikri verdi: ana yığından çıkan eleman en küçükse, yardımcı yığının tepesindeki elemana eşit olmalıdır. O halde eşitlik kontrolü yapılır, eşitse yardımcı yığından da çekilir.

- `pop` (-5): yardımcı yığının tepesine eşittir, oradan da çekilir. Ana yığın 3, 5, 1, 2, 4; yardımcı yığın 3, 1. En küçük 1.
- `pop` (4): yardımcı yığının tepesi (1) ile eşit değildir, yani 4 en küçük olamaz. Yardımcı yığın değişmez. En küçük yine 1.

Özetle:

- `push x`: `x` ana yığına itilir. Yardımcı yığının tepesindeki elemandan küçükse (ya da yardımcı yığın boşsa) yardımcı yığına da itilir.
- `pop`: Ana yığından çekilen eleman yardımcı yığının tepesine eşitse en küçük eleman çıkarılmış demektir; ikinci en küçükle devam etmek için yardımcı yığından da çekilir.

Böylece yığın var olduğu sürece herhangi bir andaki en küçük eleman tutulmuş olur.

Soru: 3 itildikten sonra 1 gelince, 1 daha küçük olduğu için 3'ü yardımcı yığından çekebilir miyiz?

Cevap: Hoca bu soruya doğrudan cevap vermedi, "o soruya geleceğim" diyerek önce `pop` durumunda en küçüğün nasıl korunacağını konuştu. Kurulan çözümde yardımcı yığından yalnız `pop` sırasında, çekilen eleman tepeye eşitse çekme yapılır; 1 geldiğinde 3 yardımcı yığında kalır.

Soru: Bu problem yardımcı yığın kullanmadan, tek bir yığınla çözülebilir mi?

Cevap: Hoca bunun da bir karşılığı olduğunu, ama bu derste konuşmayacağını, üzerine düşünülebileceğini söyledi. (Bir çözüm, Örnek Sorular bölümünde verilmiştir.)

> **Ders sonrası not:** Yardımcı yığına yalnız "küçükse" itme kuralı, aynı en küçük değer birden çok kez geldiğinde hata verir. 1 ve 1 itildiğinde yardımcı yığına yalnız ilk

1 girer. Bir pop yapılınca çekilen 1, yardımcı yığının tepesine eşit olduğu için oradan da çekilir ve yardımcı yığın boşalır; oysa ana yığında hâlâ 1 vardır. Kural “küçük ya da eşitse it” olmalıdır. Bu durum C ile denendi: “küçükse” kuralında pop sonrası en küçük bilgisi kaybolur, “küçük ya da eşitse” kuralında 1 olarak kalır. Derste kullanılan 3, 5, 1, 2, 4, -5 dizisinde tekrar eden değer olmadığı için iki kural aynı sonucu verir.

10. Bir Diziyi İki Yığın Olarak Kullanma

Hoca son olarak şu problemi verdi: elimizde n elemanlı bir dizi, yani kesintisiz bir bellek bölgesi (örneğin 100 elemanlı) var. Bu diziyi tek bir yığın olarak değil, iki yığın olarak kullanmak istiyoruz. En verimli şekilde nasıl kullanırız?

Bir öğrenci, iki yığının eleman sayısının bilinip bilinmediğini sordu. Hoca yalnız dizinin boyutunun (n) bilindiğini, iki yığın hakkında bilgi olmadığını, bir önceki problemdeki gibi birinin daha çok, diğerinin daha az kullanılabileceğini söyledi. Öğrenci bir yığını dizinin başından, diğerini sonundan başlatıp ortaya kadar büyütmeyi önerdi; iki tarafın top’u buna göre belirlenecekti.

Hoca, başlangıç olarak bunun doğru bir nokta olduğunu söyledi: bir yığının başlangıç gözü 0, diğerinin $n-1$ olur ve eleman geldikçe yerleştirilir. Ama bu fikirde verimliliği düşüren bir kısıt vardır: sınırın tam ortaya konması. Bir öğrencinin önerdiği düzeltme şudur: iki yığın birbirine çarpana kadar ortayı geçebilsin.

Hocaya göre işin püf noktası budur. Ortaya kadar kullandırılırsa bir taraf çok az alan kullanırken, diğer tarafın ihtiyacı olduğu halde öbürünün boş bıraktığı alanı kullanması engellenir. $n/2$ üzerinden gidilirse sağda ya da solda boşluklar kalır. Doğrusu, iki taraftan ilerleyen göstericilerin (top1 ve top2) birbirinin üzerinden geçmemesidir. Biri bir noktaya kadar gelmişse diğeri tam orada durmalıdır; birbirlerinin sınırı olmalıdırlar ve gerçekleştirme buna göre yapılmalıdır. Hoca bu mantıkla yapılan gerçekleştirmenin verimlilik artışı sağlayacağını söyledi.

```
gözler:   0    1    2   ...                  n-2  n-1
         [1. yığın büyür -->      <-- 2. yığın büyür]
          top1 sağa ilerler        top2 sola ilerler
          dolu: top1 ile top2 birbirini geçmek üzereyken
```

> **Hocanın vurgusu:** Bu tür yığın ve kuyruk soruları teknik iş görüşmelerinde, özellikle yurt dışında, çok sık soruluyor. Bu veri yapılarını kullanarak farklı fikirleri gerçekleyebilmek ve “zorunda kalsaydınız ne yapardınız” sorularına cevap verebilmek fark yaratır.

Örnek Sorular

Bu ders için hazır kare ve video yoktur. Aşağıdaki sorular yalnız derste sözlü olarak sorulan ya da alıştırma olarak verilen sorulardır.

Soru 1 (derste sözlü, 00:29:04)

Derleyici bir ifadedeki parantezlerin dengeli olup olmadığını yığınla nasıl kontrol eder? Hangi kurallar gözetilir? Şu örnekleri değerlendirin: (((,) ile başlayan ifade, (( ), (( )).

Çözüm (derste verildi): Her (için push, her) için pop yapılır. Boş yığında pop istenirse ya da ifade bittiğinde yığın boş değilse sözdizimi hatası vardır. (((sonda dolu kalır (hata),) ile başlayan ifade boş yığında pop ister (hata), (( ) sonda bir eleman bırakır (hata), (( )) sonda boştur (doğru). Kod ve çıktısı Bölüm 3’tedir.

Soru 2 (derste sözlü, 00:39:02)

Dizi tabanlı yığında top 0’dan başlıyorsa ve MAX 10 ise top’un alabileceği en büyük değer 9 mudur?

Çözüm (derste verildi): Bu gerçekleştirmede top, bir sonraki boş gözü ve aynı zamanda eleman sayısını gösterir. Hiç eleman yokken 0’dır, 10 eleman konunca 10 olur ve isFull doğru

döner. Dolayısıyla en büyük değer 10'dur. Derlenen programda 11 `push` denemesinden 10'u başarılı oldu ve `top` 10'da kaldı.

Soru 3 (derste sözlü, 01:04:43)

`pop` fonksiyonu çekilen değeri işaretçi üzerinden yazmak yerine doğrudan `return` ile döndürse sorun olur mu?

Çözüm (derste verildi): Yığında 0 değeri saklanabiliyorsa, dönen 0'ın "boş yığın" hatası mı yoksa çekilen değer mi olduğu ayırt edilemez. 0'ın hiç saklanmayacağı biliniyorsa sakıncası yoktur.

Soru 4 (derste sözlü, 01:09:27)

Yığını bağlı listeyle gerçekleştirirken `pop` sırasında önceki elemanı bilmek için çift yönlü bağlı liste gerekir mi? Yeni eleman listenin başına mı sonuna mı eklenmelidir?

Çözüm (derste verildi): Yeni eleman başa eklenirse tek yönlü liste yeterlidir: `push` ve `pop` yalnız `head` üzerinde yapılır. 3, 4, 2 itilince liste `head -> 2 -> 4 -> 3 -> NULL` olur ve `pop` sırasıyla 2, 4, 3 verir. Sona ekleme yapılırsa her seferinde sona gitmemek için `tail` tutulmalıdır. Kod ve çıktısı Bölüm 5'tedir.

Soru 5 (derste sözlü, 01:24:22)

`A + B * C - D / E * F` ifadesinde ilk işlem `B * C`'dir. İkinci işlem `A + B * C` yerine `D / E` olabilir mi?

Çözüm (derste verildi): Olabilir. `B * C` ile `D / E` birbirinden bağımsızdır, çevrelerinde toplama ve çıkarma vardır. Hangisinin önce yapıldığı sonucu değiştirmez.

Soru 6 (derste sözlü, 01:46:31 ve 01:47:57)

`3 * 5 + 4 / 1 - 2` ifadesini yığınla postfix'e çevirirken `+` geldiğinde (tepede `*`) ve `-` geldiğinde (tepede `/`) ne yapılır? Elde edilen iki ara değerle hangi işlem yapılır?

Çözüm (derste verildi, ders sonrası düzeltmeyle): `+` gelince tepedeki `*` daha öncelikli olduğu için çekilir, sonra `+` itilir. `-` gelince tepedeki `/` çekilir. Derste `-` daha sonra `+`'nın üzerine itildi ve çıktı `3 5 * 4 1 / 2 - +` oldu; öğrenci hesabı "15 ve 4 elde edilir, 4'ten 2 çıkarılır, sonra toplanır" diye tamamladı. Doğru kural eşit öncelikte de çekmeyi gerektirdiği için `+` da çekilmelidir; doğru postfix `3 5 * 4 1 / + 2 -`'dir ve sonuç $15+4-2=17$'dir (Bölüm 7.2'deki ders sonrası nota bakın).

Soru 7 (derste sözlü, 01:52:46)

`5 * (4 / 2 + 3 * 6) - 7` ifadesini operand ve operatör yığınlarıyla hesaplarken `+` geldiğinde `pop` mu `push` mu yapılır?

Çözüm (derste verildi): Önce `pop`: tepedeki `/`, `+`'dan daha öncelikli olduğu için çekilir; operand yığınından 2 ve 4 çekilir, $4/2=2$ itilir. Sonra `+` itilir. İfadenin tamamı 93 verir. İz ve kod Bölüm 8'dedir.

Soru 8 (derste sözlü, 01:59:02)

Öncelik gözetmeden soldan sağa hesaplamanın neden yanlış olduğunu bir örnekle gösterin.

Çözüm (derste verildi): `2 * 4 - 8 / 2 + 3` ifadesi öncelikle $8-4+3=7$ verir. Soldan sağa $2\cdot 4=8$, $8-8=0$, $0/2=0$, $0+3=3$ verir. Aynı şekilde `3 + 5 * 2 / 3 + 7` öncelikle $\approx 13{,}33$, sırayla $\approx 12{,}33$ verir.

Soru 9 (derste sözlü, 02:00:42)

Infix'ten postfix'e dönüşümün ve postfix ifadenin hesaplanmasının akış diyagramını ya da kodunu yazın.

> **Çözüm (derste verilmedi):** Hoca bunu öğrencilere alıştırma olarak bıraktı. Aşağıdaki kod, Bölüm 7'deki kuralları (eşit öncelikte de çekme düzeltmesiyle) tek karakterli operandlar için gerçekler. `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırıldı; derste geçen bütün ifadeler için çıktılar aşağıdadır.

```c
#include <stdio.h>
#include <ctype.h>
#define MAX 100

int oncelik(char op)
{
    if (op == '*' || op == '/')
        return 2;
    if (op == '+' || op == '-')
        return 1;
    return 0;                           /* '(' yığında hiçbir operatörü durdurmaz */
}

/* Tek karakterli operandlar (rakam ya da harf) için infix -> postfix */
void infixToPostfix(const char *infix, char *postfix)
{
    char yigin[MAX];
    int top = 0, k = 0;
    char c;

    for (; (c = *infix) != '\0'; infix++) {
        if (c == ' ')
            continue;
        if (isalnum((unsigned char)c)) {
            postfix[k++] = c;           /* operand doğrudan çıktıya */
        } else if (c == '(') {
            yigin[top++] = c;           /* '(' her zaman push */
        } else if (c == ')') {
            while (top > 0 && yigin[top - 1] != '(')
                postfix[k++] = yigin[--top];
            top--;                      /* '(' atılır, çıktıya yazılmaz */
        } else {
            /* tepedeki operatörün önceliği büyük ya da eşitse pop */
            while (top > 0 && oncelik(yigin[top - 1]) >= oncelik(c))
                postfix[k++] = yigin[--top];
            yigin[top++] = c;
        }
    }
    while (top > 0)                     /* ifade bitti: yığını boşalt */
        postfix[k++] = yigin[--top];
    postfix[k] = '\0';
}

/* Postfix ifadeyi tek bir operand yığınıyla hesaplar */
double postfixHesapla(const char *postfix)
{
    double yigin[MAX], a, b;
    int top = 0;
    char c;

    for (; (c = *postfix) != '\0'; postfix++) {
```

```
        if (isdigit((unsigned char)c)) {
            yigin[top++] = c - '0';
        } else {
            b = yigin[--top];           /* ilk çekilen: sağ operand */
            a = yigin[--top];           /* ikinci çekilen: sol operand */
            switch (c) {
            case '+': yigin[top++] = a + b; break;
            case '-': yigin[top++] = a - b; break;
            case '*': yigin[top++] = a * b; break;
            case '/': yigin[top++] = a / b; break;
            }
        }
    }
    return yigin[top - 1];
}

int main(void)
{
    const char *ornek[] = {
        "A+B", "A+B*C", "A+B*C+D", "A+B*C-D/E*F",
        "3*(5+4/1)-2", "3*5+4/1-2", "3+5*2/3+7", "2*4-8/2+3",
        "5*(4/2+3*6)-7"
    };
    char p[MAX];
    int i;

    for (i = 0; i < 9; i++) {
        infixToPostfix(ornek[i], p);
        if (isdigit((unsigned char)ornek[i][0]))
            printf("%-14s -> %-14s = %g\n", ornek[i], p, postfixHesapla(p));
        else
            printf("%-14s -> %s\n", ornek[i], p);
    }
    return 0;
}
```

Çıktı:

```
A+B            -> AB+
A+B*C          -> ABC*+
A+B*C+D        -> ABC*+D+
A+B*C-D/E*F    -> ABC*+DE/F*-
3*(5+4/1)-2    -> 3541/+*2-      = 25
3*5+4/1-2      -> 35*41/+2-      = 17
3+5*2/3+7      -> 352*3/+7+      = 13.3333
2*4-8/2+3      -> 24*82/-3+      = 7
5*(4/2+3*6)-7  -> 542/36*+*7-    = 93
```

Akış olarak özetlenirse: her karakter için operand mı, `(` mı, `)` mı, operatör mü diye bakılır; operandlar çıktıya, `(` yığına gider; `)` gelince `(`'ye kadar boşaltılır; operatör gelince tepedeki büyük ya da eşit öncelikliler çekilip gelen itilir; sonda yığın boşaltılır. Hesaplamada her operatör için iki operand çekilir, ilk çekilen sağ operanddır.

Soru 10 (derste sözlü, 02:02:39)

Elemanların zaman zaman `push`, zaman zaman `pop` edildiği bir yığında, herhangi bir anda yığındaki en küçük elemanı bulmak için bir yardımcı yığın nasıl kullanılır? 3, 5, 1, 2, 4, −5 sırayla itildikten sonra iki `pop` yapılırsa en küçük eleman her adımda nedir?

Çözüm (derste verildi): Gelen eleman yardımcı yığının tepesinden küçükse yardımcı yığına da itilir; çekilen eleman yardımcı yığının tepesine eşitse oradan da çekilir. Yardımcı yığın 3, 1, −5 olur, en küçük −5'tir. `pop` (−5) sonrası en küçük 1, `pop` (4) sonrası yine 1'dir.

Çözüm (derste verilmedi): Derste kod yazılmadı. Aşağıdaki kod, tekrar eden en küçük değerler için "küçük ya da eşitse it" düzeltmesiyle yazıldı ve `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırıldı.

```c
#include <stdio.h>
#define MAX 100

struct stack { int item[MAX]; int top; };

int  isEmpty(struct stack *s) { return s->top == 0; }
int  push(struct stack *s, int x) { if (s->top == MAX) return 0; s->item[s->top++] = x; return
int  pop(struct stack *s, int *x) { if (isEmpty(s)) return 0; *x = s->item[--s->top]; return 1
int  peek(struct stack *s, int *x) { if (isEmpty(s)) return 0; *x = s->item[s->top - 1]; retur

struct stack ana, yard;   /* ana yığın ve yardımcı (auxiliary) yığın */

void minPush(int x)
{
    int t;
    push(&ana, x);
    if (!peek(&yard, &t) || x <= t)   /* küçük ya da eşitse yardımcıya da it */
        push(&yard, x);
}

int minPop(int *x)
{
    int t, cop;
    if (!pop(&ana, x))
        return 0;
    if (peek(&yard, &t) && *x == t)    /* çekilen en küçükse yardımcıdan da çek */
        pop(&yard, &cop);
    return 1;
}

int enKucuk(int *m) { return peek(&yard, m); }

int main(void)
{
    int dizi[] = { 3, 5, 1, 2, 4, -5 }, i, x, m;
    for (i = 0; i < 6; i++)
        minPush(dizi[i]);
    printf("yardimci yigin:");
    for (i = 0; i < yard.top; i++) printf(" %d", yard.item[i]);
    enKucuk(&m); printf("  en kucuk = %d\n", m);
    minPop(&x); enKucuk(&m); printf("pop %d -> en kucuk = %d\n", x, m);
    minPop(&x); enKucuk(&m); printf("pop %d -> en kucuk = %d\n", x, m);

    ana.top = yard.top = 0;          /* tekrarlı en küçük: 1, 1 */
    minPush(1); minPush(1); minPop(&x);
    printf("1,1 push + pop -> en kucuk var mi: %d\n", enKucuk(&m) ? m : -999);
    return 0;
}
```

Çıktı:

```
yardimci yigin: 3 1 -5  en kucuk = -5
pop -5 -> en kucuk = 1
pop 4 -> en kucuk = 1
1,1 push + pop -> en kucuk var mi: 1
```

Aynı kodda koşul `x < t` yapılınca son satır `-999` (en küçük bilgisi yok) yazdırdı; bu, Bölüm 9'daki ders sonrası notu doğrular.

Soru 11 (derste sözlü, 02:11:10)

Aynı problem (anlık en küçük eleman) yardımcı yığın kullanmadan, tek bir yığınla çözülebilir mi?

> **Çözüm (derste verilmedi):** Hoca bu soruyu cevaplamadı. Bir çözüm, yığının her gözünde değerin yanında "o göze kadar olan en küçük değeri" de saklamaktır. `push x` sırasında yeni gözün en küçüğü, `x` ile bir alttaki gözün en küçüğünün küçüğüdür. `pop` yalnız tepedeki gözü atar; alttaki gözler kendi en küçüklerini zaten taşıdığı için ek iş gerekmez. En küçük eleman her zaman tepedeki gözde okunur. Bu çözüm de ek bellek kullanır, ama tek bir yığın yapısı içindedir. Aşağıdaki kod `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırıldı ve derstekiyle aynı sonuçları verdi.

```c
#include <stdio.h>
#define MAX 100

/* Tek yığın: her gözde değer ve o ana kadarki en küçük birlikte saklanır */
struct eleman { int deger; int enKucuk; };
struct stack { struct eleman item[MAX]; int top; };

int push(struct stack *s, int x)
{
    if (s->top == MAX)
        return 0;
    s->item[s->top].deger = x;
    if (s->top == 0 || x < s->item[s->top - 1].enKucuk)
        s->item[s->top].enKucuk = x;
    else
        s->item[s->top].enKucuk = s->item[s->top - 1].enKucuk;
    s->top++;
    return 1;
}

int pop(struct stack *s, int *x)
{
    if (s->top == 0)
        return 0;
    *x = s->item[--s->top].deger;
    return 1;
}

int enKucuk(struct stack *s, int *m)
{
    if (s->top == 0)
        return 0;
    *m = s->item[s->top - 1].enKucuk;
    return 1;
}

int main(void)
{
    struct stack s = { .top = 0 };
    int dizi[] = { 3, 5, 1, 2, 4, -5 }, i, x, m;
    for (i = 0; i < 6; i++)
        push(&s, dizi[i]);
    enKucuk(&s, &m); printf("en kucuk = %d\n", m);
    pop(&s, &x); enKucuk(&s, &m); printf("pop %d -> en kucuk = %d\n", x, m);
    pop(&s, &x); enKucuk(&s, &m); printf("pop %d -> en kucuk = %d\n", x, m);
    return 0;
}
```

Çıktı:

```
en kucuk = -5
pop -5 -> en kucuk = 1
pop 4 -> en kucuk = 1
```

Soru 12 (derste sözlü, 02:11:43)

n elemanlı tek bir dizi iki yığın olarak en verimli nasıl kullanılır?

Çözüm (derste verildi): Bir yığın 0. gözden sağa, diğeri $n-1$. gözden sola büyür. Sınır ortaya sabitlenmez; yığınlar birbirine değene kadar büyüyebilir. Tek koşul, `top1` ile `top2`'nin birbirinin üzerinden geçmemesidir. Böylece bir yığının kullanmadığı alanı diğeri kullanabilir.

> **Çözüm (derste verilmedi):** Derste kod yazılmadı. Aşağıdaki kodda iki `top` da bir sonraki boş gözü gösterir; `top1 > top2` olduğunda boş göz kalmamıştır. Örnekte $n = 6$ alındı: birinci yığın 5 eleman kullanınca ikinci yığına yalnız bir yer kalır; birinci yığından bir `pop` yapılınca o göz ikinci yığına geçer. `gcc -Wall -Wextra` ile uyarısız derlenip çalıştırıldı.

```c
#include <stdio.h>
#define N 6

int dizi[N];
int top1 = 0;        /* 1. yığın: 0. gözden sağa büyür, bir sonraki boş gözü gösterir */
int top2 = N - 1;    /* 2. yığın: n-1. gözden sola büyür, bir sonraki boş gözü gösterir */

int doluMu(void) { return top1 > top2; }   /* boş göz kalmadı */

int push1(int x) { if (doluMu()) return 0; dizi[top1++] = x; return 1; }
int push2(int x) { if (doluMu()) return 0; dizi[top2--] = x; return 1; }
int pop1(int *x) { if (top1 == 0) return 0; *x = dizi[--top1]; return 1; }
int pop2(int *x) { if (top2 == N - 1) return 0; *x = dizi[++top2]; return 1; }

int main(void)
{
    int i, x, n1 = 0, n2 = 0;
    for (i = 1; i <= 5; i++) n1 += push1(i * 10);   /* 1. yığın çok kullanılıyor */
    n2 += push2(99);
    n2 += push2(98);                                /* yer kalmadı */
    printf("push1 basarili: %d, push2 basarili: %d, top1 = %d, top2 = %d\n",
           n1, n2, top1, top2);
    pop1(&x); printf("pop1 -> %d\n", x);
    printf("push2(98) simdi: %d\n", push2(98));
    pop2(&x); printf("pop2 -> %d\n", x);
    pop2(&x); printf("pop2 -> %d\n", x);
    printf("bos 2. yiginda pop2: %d\n", pop2(&x));
    return 0;
}
```

Çıktı:

```
push1 basarili: 5, push2 basarili: 1, top1 = 5, top2 = 4
pop1 -> 50
push2(98) simdi: 1
pop2 -> 98
pop2 -> 99
bos 2. yiginda pop2: 0
```

Ortadan bölünmüş bir tasarımda birinci yığın en fazla 3 eleman alabilirdi; burada 5 eleman alabildi.

Dersin Toparlaması

Ders, bağlı listenin bir bütün olarak değerlendirilmesiyle başladı. Dinamik yapı, kaydırmasız ekleme-silme, esnek bellek kullanımı ve ölçeklenebilirlik avantajlarına karşılık ek adres maliyeti, doğrudan erişim ve ikili arama olmaması, gerçekleştirme karmaşıklığı ve özellikle önbellek verimsizliği dezavantajları konuşuldu. Ardından yığın soyut veri tipine geçildi. LIFO mantığı, kullanım alanları (ifade dönüşümü, özyineleme, dizgi ters çevirme, geri izleme, parantez kontrolü, taban dönüşümü, geri al) ve temel işlemler (`initStack`, `isEmpty`, `isFull`, `push`, `pop`, `peek`, `size`) görüldü. Yığın önce diziyle, `top`'un hem bir sonraki boş gözü hem eleman sayısını gösterdiği bir gerçekleştirmeyle kuruldu. Sonra yeni elemanların başa eklendiği tek yönlü bağlı listenin yığın için yeterli olduğu gösterildi.

Dersin ikinci yarısı aritmetik ifadelere ayrıldı. Bilgisayarın infix ifadeyi soldan sağa işlerse işlem önceliğini bozacağı, bu yüzden ifadenin postfix'e çevrildiği anlatıldı ve dönüşüm parantezli ve parantezsiz örneklerle yığın üzerinde adım adım yapıldı. Derleyicilerin dönüşüm ve hesabı operand ve operatör yığınlarıyla aynı anda yaptığı `5 * (4 / 2 + 3 * 6) - 7 = 93` örneğiyle gösterildi. Son olarak iki iş görüşmesi tipi problem tartışıldı: yardımcı yığınla anlık en küçük elemanı takip etmek ve tek bir diziyi iki yığına verimli paylaştırmak. Hoca, dönüşüm ve değerlendirme algoritmalarının ve en küçük eleman probleminin akış diyagramlarını ya da kodlarını haftaya kadar yazarak alıştırma yapmayı önerdi. Gelecek hafta yığınla ilgili birkaç örnek daha çözülecek, belki bağlı listeye dair birkaç örnek daha yapılacak, ardından kuyruk (queue) konusuna geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.